

异步在线评测系统

Async OJ · 课程实验报告

系统定位	面向程序设计训练的安全、可追踪、可扩展在线评测平台
核心关键词	FastAPI · Streamlit · asyncio · SQLite · Bubblewrap · AI 命题
报告重点	系统功能与设计 / 关键实现与难点 / 成果展示与边界测试

实验结论摘要

本系统完成了从用户认证、题目维护、代码提交、异步评测到结果审计的闭环，并将不可信代码放入 Bubblewrap 隔离环境中执行。项目测试集覆盖后端、前端、评测器、沙箱、权限和 AI 命题质量门槛；在当前虚拟环境中验证结果为 92 项通过、1 项跳过，Ruff 静态检查通过。

一、系统功能与设计

1.1 系统架构

系统采用前后端分离架构。Streamlit 只负责交互与页面状态，所有业务数据通过 REST API 访问；FastAPI 路由统一使用 `async def`，服务层负责业务编排，SQLite 与题目 JSON 文件分别承担运行状态和题库配置持久化。评测与 AI 命题以后台任务运行，避免阻塞 HTTP 请求。

REST API 是前后端唯一边界；后台任务通过 `asyncio` 调度，重启后恢复 pending 评测

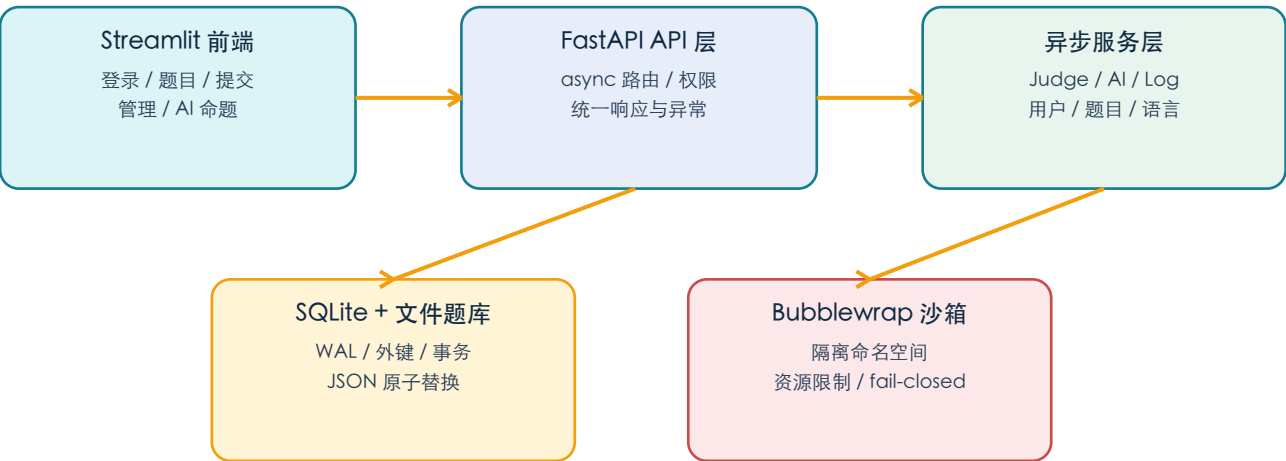


图 1 系统总体架构与数据边界

1.2 主要功能

功能域	用户侧能力	设计要点
认证与用户	注册、登录、登出、个人信息；管理员管理角色	bcrypt-SHA256 密码摘要；数据库 Session + 过期时间；封禁立即撤销 Session
题目管理	题目列表/详情；新建、编辑、删除；资源限制和日志可见性	每题一个 UTF-8 JSON；临时文件 + 原子替换；删除仅管理员
在线评测	提交 Python/C++14 代码；刷新状态；查看分数与测试点日志；管理员重判	pending -> 后台调度；AC/WA/RE/TLE/MLE/CE；问题与语言双重资源限制
审计与可见性	按用户/题目查询日志访问记录	私有题目只给本人总分；详情访问成功/拒绝均写入审计
AI 智能命题	模型配置、习惯配置、异步生成、反馈修订、导入题目	最多 10 个用户隔离习惯；结构校验 + 质量门槛 + 一次自动修复；版本链
前端体验	侧边栏导航、表单化题目录入、状态刷新、友好错误提示	Session 按 Streamlit 会话隔离；仅通过 <code>api_client.py</code> 调用后端

表 1 主要功能与设计取舍

1.3 技术选型与模块划分

层次	技术/模块	职责
表示层	Streamlit / frontend.ui.py / forms.py / ai_page.py	页面、交互、表单校验、会话状态与错误展示
接口层	FastAPI / app.api.routes / dependencies	REST API、统一响应、认证依赖、角色权限
业务层	app.services	用户、题目、评测、沙箱、日志、AI、系统重置
数据层	aiosqlite / repositories / problems/*.json	异步 SQLite、外键级联、题目文件原子写入
执行层	asyncio subprocess / psutil / Bubblewrap	编译、运行、超时与内存监控、进程组回收、隔离
质量保障	pytest + pytest-asyncio + Ruff	93 项自动化测试、异步行为与静态规范检查

二、关键实现与难点

2.1 异步评测闭环

提交接口只创建记录并立即返回 pending，judge_tasks.py 负责调度后台任务。JudgeService 对每个测试点依次执行：创建独立临时目录、编译（如需要）、注入输入、收集输出、规范化换行和末尾空白、比对标准输出，并将结果、耗时、峰值内存写入 testcase_results。服务启动时会扫描 pending 记录并恢复评测，避免重启导致任务永久丢失。

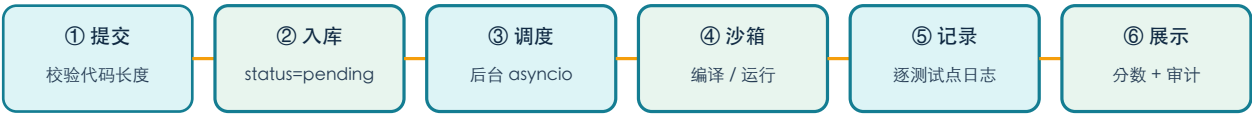


图 2 提交到结果展示的异步流水线

2.2 安全沙箱：不可信代码的多层约束

沙箱是本系统最关键的工程难点。Bubblewrap 命令显式关闭 user、PID、IPC、网络、UTS、cgroup 等命名空间，清空环境变量，删除全部 Linux capability，只读挂载必要系统目录，将提交目录映射为 /workspace；同时结合 RLIMIT_AS、RLIMIT_CPU、RLIMIT_FSIZE、RLIMIT_NOFILE、RLIMIT_NPROC 和 psutil 监控，控制内存、CPU、文件大小、文件描述符、进程数和输出。启动自检失败、运行在 root 或非 Linux 环境时，安全模式直接拒绝服务启动（本地可信开发可显式关闭）。

2.3 动态语言配置的安全边界

语言注册并非简单保存命令模板：只允许配置白名单中的可执行文件名，拒绝 shell 控制字符、外部路径、内联代码、编译器插件和命令包装器；{src} 与 {exe} 必须作为独立参数。实际执行前还会重新校验数据库中的命令，防止旧配置绕过当前注册规则。这个“双阶段验证”解决了配置持久化后信任边界变弱的问题。

2.4 AI 命题的质量门槛与版本链

AI 命题不是“生成即通过”：后端检查 JSON 结构、样例/测试点独立性、输入去重、覆盖目的去重、边界场景数量、大规模场景和输入规模层次，并拒绝低信息量的重复长输入。首次不合格会携带具体反馈自动修复一次；仍不合格

则失败。每个通过验证的修订版本保存父版本、反馈、完整结果和用量，支持从历史版本分支继续修改。API Key 只留在后端进程内存中，不写入数据库或响应。

2.5 权限、日志与可见性

普通用户、提交者本人、管理员三类访问边界由 LogService 集中判断。私有题目下，提交者仅看到总分，管理员看到逐测试点详情；题目公开后，已登录用户也可查看详情。无论访问成功还是被拒绝，日志接口都会记录审计状态，从而保证“谁在何时查看了哪道题的评测日志”可追踪。

三、成果展示

3.1 真实运行页面

以下截图来自本项目实际运行页面，展示管理员对用户数据、角色和 AI 智能命题任务的管理能力。截图中的用户数量、提交次数和任务 Token 用量均来自运行时页面，可作为系统效果与功能闭环的直观证据。

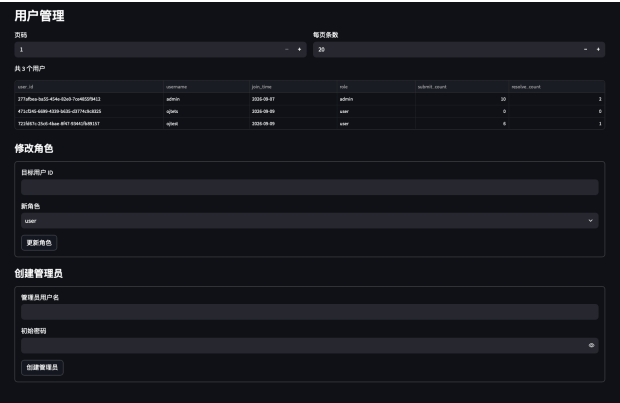


图 3 用户管理页面：分页用户列表、提交/通过统计、角色修改与创建管理员

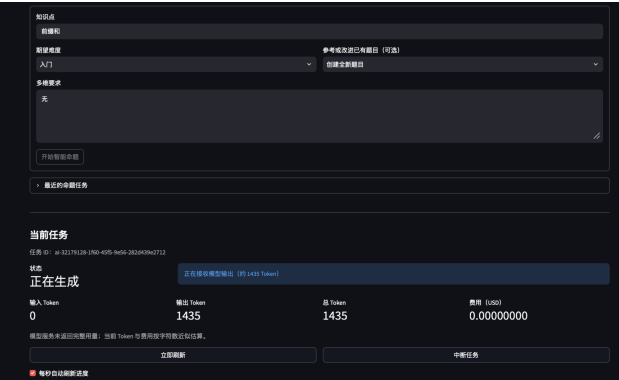


图 4 AI 智能命题页面：知识点、难度、多维要求、实时生成状态与 Token/费用统计

从图 3 可见，管理员能够查看用户 ID、注册日期、角色以及提交/通过数量，并可通过表单修改角色或创建管理员；这与认证、角色权限和审计模块形成对应。图 4 展示了 AI 命题的输入区和当前任务区：用户选择知识点、期望难度和改题对象后，系统以异步任务方式持续反馈生成状态、Token 用量和费用估算，体现了前端、后台任务、模型服务和质量验证之间的联动。

3.2 自动化测试与边界结果

测试域	覆盖内容	结果
基础与 API	健康检查、统一响应 envelope、错误码一致性、所有路由 async	通过
认证与权限	注册/登录/登出、过期 Session、管理员创建、角色变更、封禁	通过
题目与持久化	增删改查、重复/非法配置、JSON 原子写入、重启持久化、删除级联	通过
评测器	Python AC/WA/RE/TLE/MLE、输出超限、代码长度、C++ 编译/编译错误、重判	通过
沙箱安全	Bubblewrap 参数、缺失可执行文件、root 拒绝、网络/主机文件/环境隔离	通过（真实沙箱项按环境跳过 1 项）
日志审计	私有/公开测试点、本人/管理员权限、成功与拒绝审计、分页筛选	通过
AI 命题	模型配置脱敏、习惯隔离、质量门槛、自动修复、版本修订、取消	通过
前端	Cookie 会话、错误提示、题目表单、测试点结果、审计展示、API 边界	通过

测试域	覆盖内容	结果
汇总	93 项 pytest ; Ruff 静态检查	92 passed / 1 skipped ; All checks passed

表 2 自动化测试结果（当前 .venv 环境）

3.3 边界测试矩阵

边界输入/场景	预期行为	设计保障
代码超过题目长度限制	请求被拒绝；已存提交重判前再次校验	提交入口 + JudgeService 双重校验
程序超时 / 内存超限 / 输出过大	分别返回 TLE / MLE / 受控错误，不拖垮服务	超时 wait_for、进程组 kill、psutil、bounded stream
题目测试点重复或覆盖目的笼统	AI 草稿不通过并自动修复一次	输入规范化、去重、关键词和规模层次门槛
普通用户查看他人私有日志	403，且写入拒绝审计记录	LogService 集中授权判断 + access_logs
Bubblewrap 缺失 / root 启动 / 非 Linux	安全模式拒绝启动，不降级执行不可信代码	启动自检与 fail-closed 策略
服务重启时存在 pending 评测	恢复调度；AI running/pending 标记失败	lifespan 启动恢复与关闭清理

3.4 实验总结

本实验的主要成果不仅是完成 OJ 的基本“提交-判题-出分”功能，还建立了可扩展的工程边界：前后端隔离让界面与业务解耦，异步调度让评测与 AI 生成不阻塞请求，Bubblewrap 和资源限制把不可信代码执行约束在独立环境，日志可见性与访问审计补齐了教学系统中的数据治理要求。后续可继续扩展更多语言模板、题目统计和部署监控，但当前核心功能与关键边界已由自动化测试覆盖。

附录：项目结构与运行说明

项目入口为 `app.main:create_app` 与 `streamlit_app.py`。后端默认使用 `http://127.0.0.1:8000`，前端通过 `OJ_API_URL` 或侧边栏修改 API 地址；生产评测环境需要 Linux、Python 3.10+、GCC 9+、C++14 和 Bubblewrap。开发验证可运行 `.venv/bin/pytest -q` 与 `.venv/bin/ruff check .`。

目录	内容
<code>app/api</code>	FastAPI 路由、依赖和统一 API 入口
<code>app/services</code>	认证、用户、题目、评测、沙箱、日志、AI 等业务服务
<code>app/db</code>	SQLite schema、初始化、默认管理员和默认语言
<code>frontend</code>	Streamlit 页面、表单构造和 API 客户端
<code>tests</code>	93 项异步/单元/前端行为/安全边界测试
<code>problems</code>	每道题一个 UTF-8 JSON 配置文件

报告依据：项目源码、README.md、tests/ 测试用例及当前虚拟环境的 `pytest` / `Ruff` 执行结果。